Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-207БВ-25

Студент: Михеева А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 07.11.26

Москва, 2026

Постановка задачи

Вариант 20.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Основной процесс должен создать для решения задачи дочерние процессы. Взаимодействие между процессами осуществляется через отображаемые файлы (memory-mapped files) и системные семафоры для синхронизации. Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает двух дочерних процессов. Родительский и дочерний процессы представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в разделяемую память с указанием, какому процессу они предназначены. Правило фильтрации: строки длиной больше 10 символов отправляются для child2, иначе — для child1. Дочерние процессы инвертируют строки и пишут результаты своей работы в стандартный вывод.

Общий метод и алгоритм решения

Использованные системные вызовы:

- int shm_open(const char *name, int oflag, mode_t mode); — открывает или создаёт именованный объект разделяемой памяти. Возвращает файловый дескриптор.
- int ftruncate(int fd, off_t length); — устанавливает размер объекта разделяемой памяти. Необходим перед отображением в адресное пространство.
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); — отображает объект разделяемой памяти в адресное пространство процесса.
- int munmap(void *addr, size_t length); — удаляет отображение памяти.
- int shm_unlink(const char *name); — удаляет имя объекта разделяемой памяти.
- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); — открывает или создаёт именованный семафор. Значение value задаёт начальное состояние счётчика.
- int sem_wait(sem_t *sem); — захватывает семафор. Уменьшает счётчик на единицу. Если счётчик равен нулю, поток блокируется.
- int sem_post(sem_t *sem); — освобождает семафор. Увеличивает счётчик на единицу.
- int sem_close(sem_t *sem); int sem_unlink(const char *name); — закрывает и удаляет семафор.
- pid_t fork(void); — создаёт дочерний процесс.
- int execl(const char *path, const char *arg, ...); — заменяет образ текущего процесса новой программой.
- pid_t waitpid(pid_t pid, int *status, int options); — ожидает завершения дочернего процесса.

В отличие от лабораторной работы №1, где взаимодействие между процессами осуществлялось через неименованные каналы (pipe), в данной работе используется именованная разделяемая память. Разделяемая память не является файлом на диске, а существует в оперативной памяти (в Linux — в виртуальной файловой системе /dev/shm). Это обеспечивает более высокую скорость обмена данными, но не предоставляет встроенных механизмов блокировки. Поскольку каналы, в отличие от разделяемой памяти, имеют встроенную синхронизацию (блокирующее чтение), для предотвращения гонки данных и корректной синхронизации процессов используются именованные семафоры.

Программа состоит из двух исполняемых файлов: parent и children.

Сначала родительский процесс (parent) генерирует уникальные имена для разделяемой памяти и двух семафоров, используя системный вызов getpid(), чтобы можно было запускать несколько экземпляров программы без конфликтов. Имена формируются функцией snprintf и имеют вид /shm_PID, /sem_empty_PID, /sem_full_PID.

Затем parent создаёт объект разделяемой памяти с помощью shm_open с флагами O_CREAT и O_RDWR, задаёт его размер через ftruncate и отображает его в своё адресное пространство с помощью mmap. Создаются два семафора: sem_empty с начальным значением 1 (память свободна, можно писать) и sem_full с начальным значением 0 (данных пока нет). Эта схема реализует классическую модель Producer-Consumer.

Далее parent дважды вызывает fork и в каждом дочернем процессе с помощью execl запускает программу children, передавая ей в качестве аргументов свой идентификатор (1 или 2), а также имена разделяемой памяти и семафоров.

Дочерние процессы подключаются к уже существующей разделяемой памяти через shm_open с флагами O_RDWR и отображают её с помощью mmap. Семафоры открываются через sem_open без флага O_CREAT. Дочерние процессы переходят в бесконечный цикл ожидания данных.

Родительский процесс считывает строки из стандартного ввода с помощью fgets. Для каждой строки определяется её длина. Если длина больше 10 символов, строка предназначена для child2 (target = 2), иначе для child1 (target = 1). Перед записью в разделяемую память parent вызывает sem_wait(sem_empty), чтобы дождаться, пока память станет свободной. Затем записывает длину, целевой процесс и сами данные в разделяемую память и вызывает sem_post(sem_full), сигнализируя о готовности данных. Если введена пустая строка или достигнут конец файла (Ctrl+D), цикл завершается, и родитель отправляет сигнал завершения.

Дочерние процессы в цикле вызывают sem_wait(sem_full). Проснувшись, каждый ребёнок проверяет, не установлен ли флаг завершения (done). Если установлен — он освобождает семафор и завершается. Если данные предназначены для него (target совпадает с его идентификатором), он копирует строку в локальный буфер, немедленно освобождает семафор sem_empty (чтобы родитель мог писать дальше), инвертирует строку и выводит её в стандартный поток вывода. Если данные предназначены для другого процесса, ребёнок возвращает сигнал, вызывая sem_post(sem_full), чтобы разбудить брата.

После завершения цикла родитель отправляет два сигнала в sem_full, чтобы разбудить обоих детей, ожидает их завершения с помощью waitpid, а затем освобождает ресурсы: munmap, close, shm_unlink, sem_close, sem_unlink.

Код программы

```
twentyeighth@connor:~/lab$ cat parent.c

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/mman.h>
```

```
#include <sys/wait.h>
#include <semaphore.h>
#include <string.h>
#include <stdbool.h>
#include <stdint.h>

#define SHM_SIZE 8192

typedef struct {
    bool from_parent;
    bool done;
    int target;
    uint32_t length;
    char data[SHM_SIZE - 32];
} SharedData;

int main() {
    char shm_name[64], sem_empty_name[64], sem_full_name[64];

    snprintf(shm_name, sizeof(shm_name), "/shm_%d", getpid());
    snprintf(sem_empty_name, sizeof(sem_empty_name), "/sem_empty_%d", getpid());
    snprintf(sem_full_name, sizeof(sem_full_name), "/sem_full_%d", getpid());

    int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0600);
    if (shm_fd == -1) {
        const char msg[] = "error: failed to create SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    if (ftruncate(shm_fd, sizeof(SharedData)) == -1) {
        const char msg[] = "error: failed to resize SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }
```

```
    SharedData* shm = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
    if (shm == MAP_FAILED) {
        const char msg[] = "error: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    sem_t* sem_empty = sem_open(sem_empty_name, O_CREAT, 0600, 1);
    sem_t* sem_full = sem_open(sem_full_name, O_CREAT, 0600, 0);

    if (sem_empty == SEM_FAILED || sem_full == SEM_FAILED) {
        const char msg[] = "error: failed to create semaphore\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    shm->done = false;

    pid_t pid1 = fork();
    if (pid1 == 0) {
        execl("./children", "children", "1", shm_name, sem_empty_name, sem_full_name,
NULL);
        const char msg[] = "error: failed to exec\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }
    else if (pid1 == -1) {
        const char msg[] = "error: failed to fork\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
```

```
        execl("./children", "children", "2", shm_name, sem_empty_name, sem_full_name,
NULL);
        const char msg[] = "error: failed to exec\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }
    else if (pid2 == -1) {
        const char msg[] = "error: failed to fork\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    char buf[SHM_SIZE];
    while (true) {
        if (!fgets(buf, sizeof(buf), stdin)) break;
        if (buf[0] == '\n' || buf[0] == '\0') break;

        size_t len = strlen(buf);
        if (len > 0 && buf[len - 1] == '\n') buf[len - 1] = '\0';
        len = strlen(buf);
        if (len == 0) break;

        int target = (len > 10) ? 2 : 1;

        sem_wait(sem_empty);

        shm->from_parent = true;
        shm->target = target;
        shm->done = false;
        shm->length = len;
        memcpy(shm->data, buf, len);

        sem_post(sem_full);
    }

    sem_wait(sem_empty);
```

```
    shm->from_parent = true;
    shm->done = true;
    shm->target = 0;
    sem_post(sem_full);
    sem_post(sem_full);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    munmap(shm, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);
    sem_close(sem_empty);
    sem_unlink(sem_empty_name);
    sem_close(sem_full);
    sem_unlink(sem_full_name);

    return 0;
}
twentyeighth@connor:~/lab$ cat children.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <string.h>
#include <stdbool.h>
#include <stdint.h>

#define SHM_SIZE 8192

typedef struct {
    bool from_parent;
    bool done;
```

```
    int target;
    uint32_t length;
    char data[SHM_SIZE - 32];
} SharedData;


int main(int argc, char** argv) {
    if (argc < 5) {
        fprintf(stderr, "usage: %s <1|2> <shm_name> <sem_empty> <sem_full>\n",
argv[0]);
        return 1;
    }

    int my_id = atoi(argv[1]);
    const char* shm_name = argv[2];
    const char* sem_empty_name = argv[3];
    const char* sem_full_name = argv[4];

    int shm_fd = shm_open(shm_name, O_RDWR, 0600);
    if (shm_fd == -1) {
        const char msg[] = "error: failed to open SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    SharedData* shm = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
    if (shm == MAP_FAILED) {
        const char msg[] = "error: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        _exit(EXIT_FAILURE);
    }

    sem_t* sem_empty = sem_open(sem_empty_name, 0);
    sem_t* sem_full = sem_open(sem_full_name, 0);
    if (sem_empty == SEM_FAILED || sem_full == SEM_FAILED) {
        const char msg[] = "error: failed to open semaphore\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg));
    _exit(EXIT_FAILURE);
}

while (true) {
    sem_wait(sem_full);

    if (shm->done) {
        sem_post(sem_empty);
        break;
    }

    if (shm->target == my_id) {
        uint32_t len = shm->length;
        char local_buf[SHM_SIZE];
        memcpy(local_buf, shm->data, len);
        local_buf[len] = '\0';

        sem_post(sem_empty);

        for (uint32_t i = 0; i < len / 2; i++) {
            char t = local_buf[i];
            local_buf[i] = local_buf[len - 1 - i];
            local_buf[len - 1 - i] = t;
        }

        printf("[Child %d] %s\n", my_id, local_buf);
        fflush(stdout);
    }
    else {
        sem_post(sem_full);
    }
}

munmap(shm, sizeof(SharedData));
```

```
    close(shm_fd);
    sem_close(sem_empty);
    sem_close(sem_full);
    return 0;
}
```

Протокол работы программы

```
twentyeighth@connor:~/lab$ ls
children  children.c  parent  parent.c
twentyeighth@connor:~/lab$ cc -o parent parent.c -lrt
twentyeighth@connor:~/lab$ cc -o children children.c -lrt
twentyeighth@connor:~/lab$ ./parent
helloworld
[Child 1] dlrowolleh
Zatanna
[Child 1] annataZ
Amaze amaze amaze
[Child 2] ezama ezama ezamA
```

Proper nouns are a headache. If you're learning German from a guy named Hans, you just call him Hans. But I literally can't make the noises Rocky makes and vice versa. So when one of us tells the other about a name, the other one has to pick or invent a word to represent that name in their own language. Rocky's actual name is a sequence of notes - he told it to me once but it has no meaning in his language, so I stuck with "Rocky." But my name is actually an English word. So Rocky just calls me the Eridian word for "grace." Anyway, now I have to come up with an English word that means "Rocky's spouse." "Adrian," I say. Why not? "Human word is 'Adrian.'"

[Child 2] "'.nairdA' si drow namuH" ?ton yhW .yas I ",nairdA" ".esuops s'ykcoR" snaem taht drow hsilgnE na htiw pu emoc ot evah I won ,yawynA ".ecarg" rof drow naidirE eht em sllac tsuj ykcoR oS .drow hsilgnE na yllautca si eman ym tuB ".ykcoR" htiw kcuts I os ,egaugnal sih ni gninaem on sah ti tub ecno em ot ti dlot eh - seton fo ecneuqes a si eman lautca s'ykcoR .egaugnal nwo rieht ni eman taht tneserper ot drow a tnevni ro kcip ot sah eno rehto eht ,eman a tuoba rehto eht sllet su fo eno nehw oS .asrev eciv dna sekam ykcoR sesion eht ekam t'nac yllaretil I tuB .snaH mih llac tsuj uoy ,snaH deman yug a morf namreG gninrael er'uoy fI .ehcadaeh a era snuon reporP

Вывод

В ходе лабораторной работы программа из первой лабораторной была переработана с использованием технологии разделяемой памяти (memory-mapped files) и семафоров. Основная сложность возникла при синхронизации процессов: в отличие от каналов, где блокировка чтения и записи происходит автоматически, разделяемая память требует ручного управления доступом. Для этого была реализована модель Producer-Consumer с двумя семафорами (на запись и на чтение). Это позволило избежать гонки данных и корректно распределять строки между дочерними процессами. В целом работа дала понимание механизмов memory mapping и синхронизации через POSIX-семафоры.